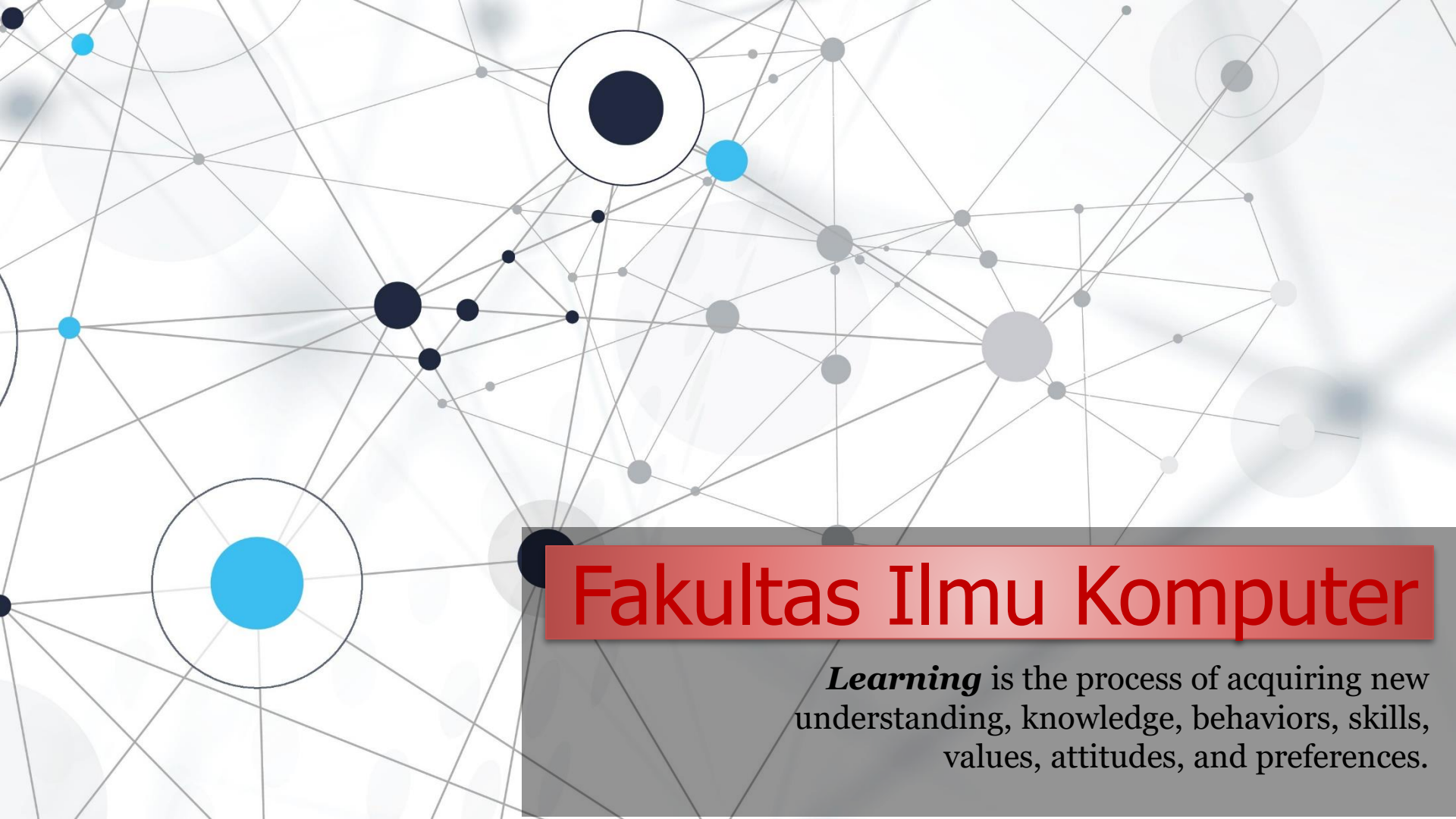


A complex network diagram with numerous nodes of varying sizes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger concentric circles. The background is light grey with faint circular patterns.

Fakultas Ilmu Komputer

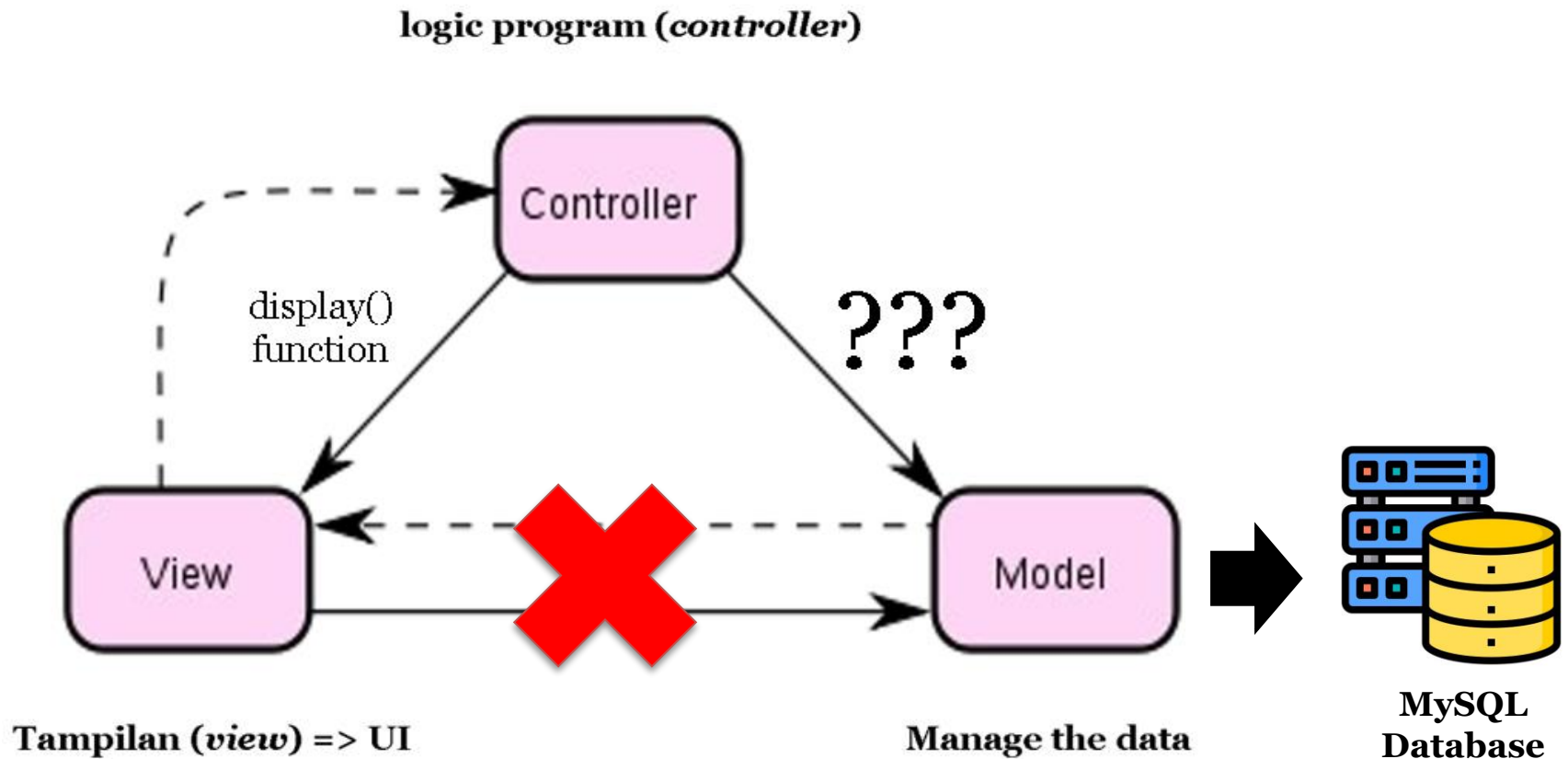
Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

Back-end Framework #7

[Model & Database Query]



Welcome to Back-end Web Development *course*



Kita sudah membuat connector antara komponen Controller dan View, apakah kita harus membuat connector antara komponen Controller dan Model???

Tugas dan Tanggung jawab **Back End Developer**

01.

Membuat program yang aman dari hacker/virus

02.

Mendesain struktur data website dan database

03.

Mengembangkan coding dan menemukan error

04.

Update Produk/Aplikasi

05.

Mengembangkan struktur data dasar

06.

Bekerjasama dengan Front-End untuk tampilan produk

07.

Mengatasi masalah pada server

Back-end developers are responsible for **manipulating data** according to the needs of the application. This could be retrieving data from the database, storing new data, modifying existing data, or deleting data. Knowledge of SQL (Structured Query Language) used in databases such as MySQL is required to perform these operations. **Back-end developers** are also responsible for **designing efficient database schemas** and **optimizing queries** for better performance.



You're Started Here !!!

A decorative border of green leaves and branches at the top of the slide.

1 – Create Model Connector in Core Controller Class

A decorative border of green leaves and branches at the bottom of the slide.

1 – Model Connector in Core Logic

```
C:\xampp\htdocs\lecture20\aimvc\app\core\Controller.php (aimvc) - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES
x Controller.php

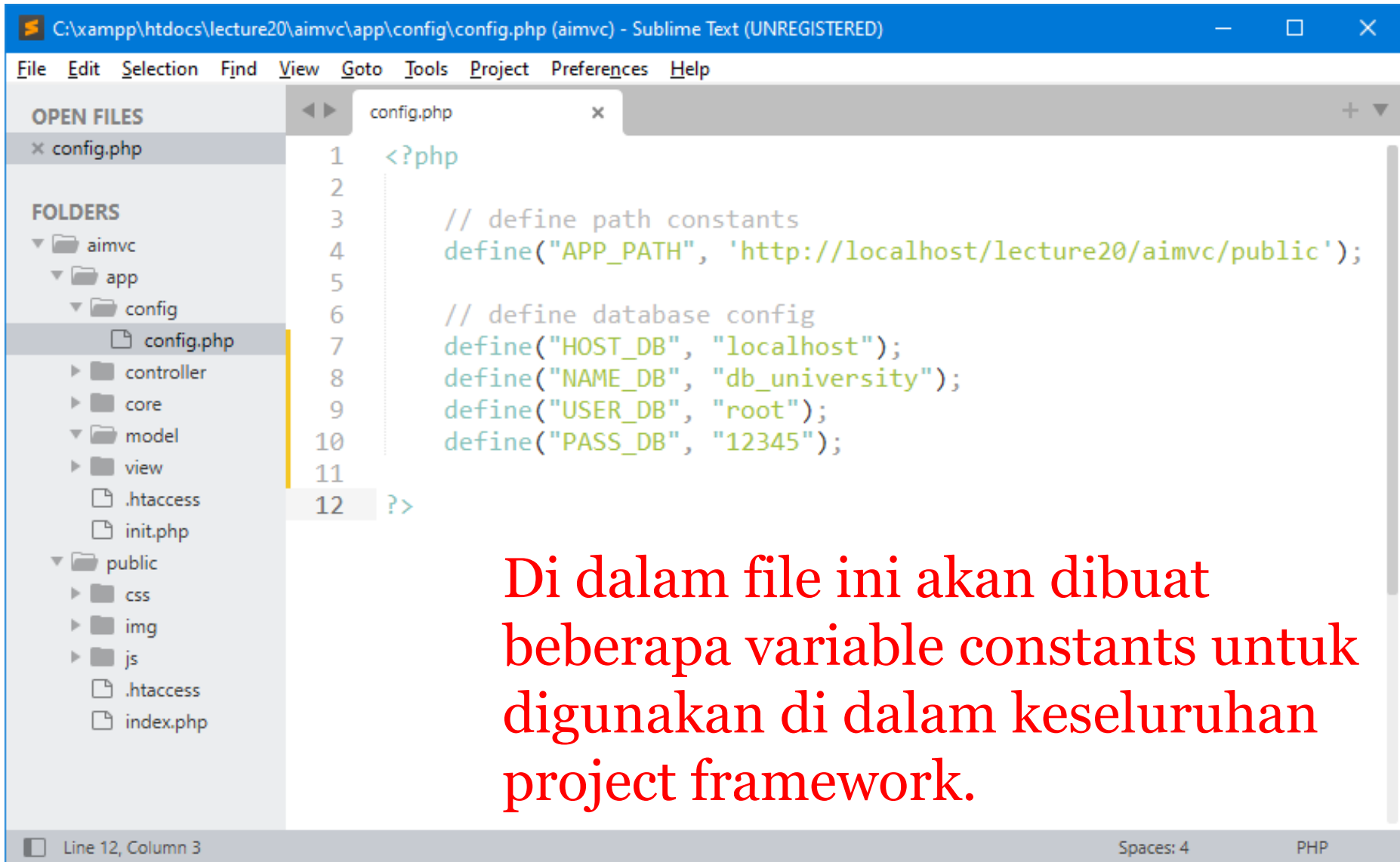
FOLDERS
aimvc
  app
    config
    controller
    core
      App.php
      Controller.php
    model
    view
    .htaccess
    init.php
  public
    css
    img
    js
    .htaccess
    index.php

1  <?php
2      // Create a class
3      class Controller{
4          // PHP Constructor
5          public function __construct(){
6              echo "Object is created.";
7          }
8
9      // display method and send any data
10     public function display($view, $data=[]){
11
12         require_once "../app/view/".$view.".php";
13
14     }
15
16     // core logic model method
17     public function logic($model){
18         require_once "../app/model/".$model.".php";
19         $obj_model = new $model;
20         return $obj_model;
21     }
22 }
23 ?>
```

Line 16, Column 1; Saved C:\xampp\htdocs\lecture20\aimvc\app\core\Controller.php (UTF-8) Tab Size: 4 PHP

Model component stores data and its ***related logic***.

2 - Config a Connection to MySQL



```
C:\xampp\htdocs\lecture20\aimvc\app\config\config.php (aimvc) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES
x config.php

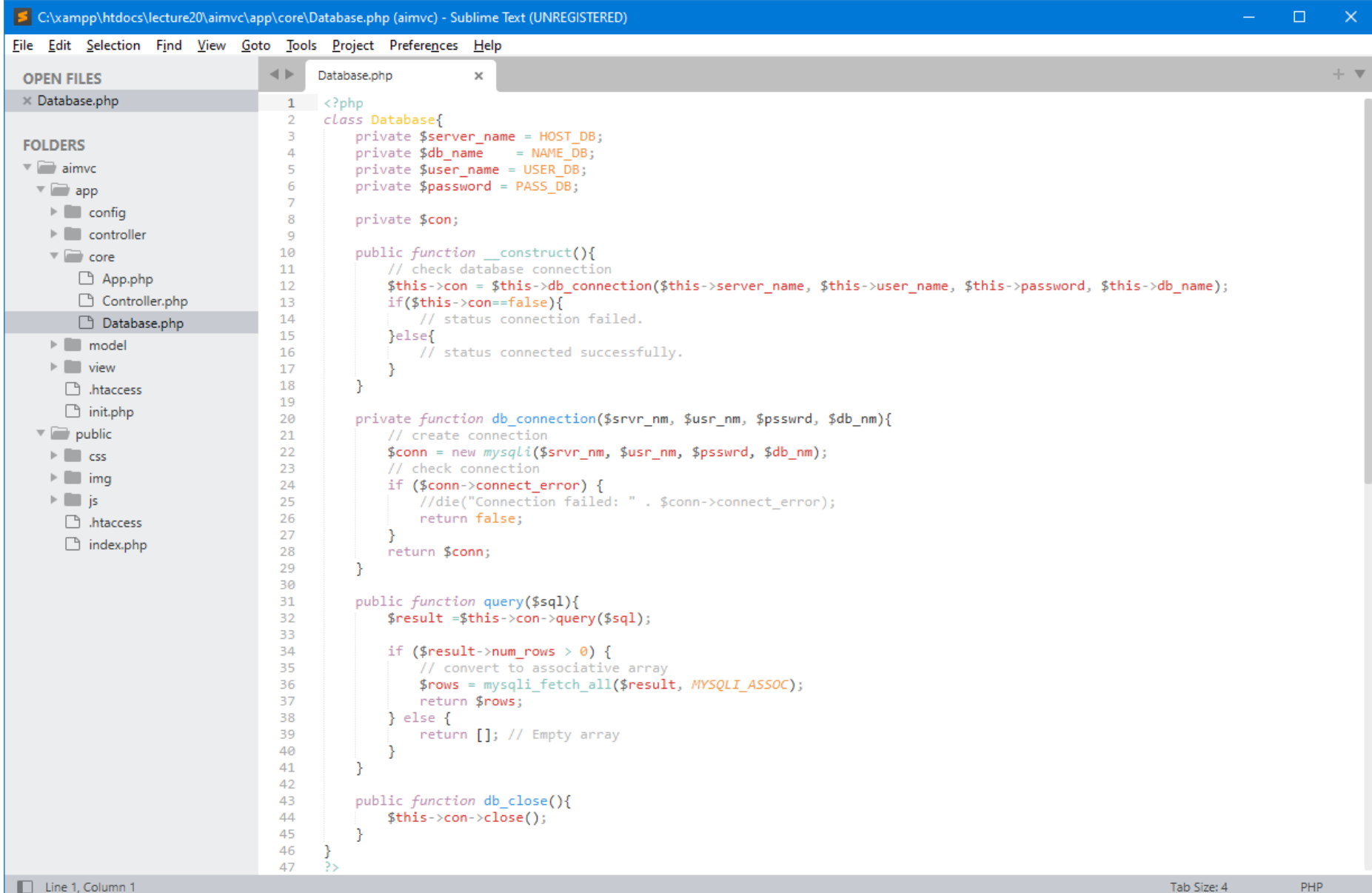
FOLDERS
v aimvc
  v app
    v config
      config.php
    controller
    core
    model
    view
    .htaccess
    init.php
  public
    css
    img
    js
    .htaccess
    index.php

1  <?php
2
3  // define path constants
4  define("APP_PATH", 'http://localhost/lecture20/aimvc/public');
5
6  // define database config
7  define("HOST_DB", "localhost");
8  define("NAME_DB", "db_university");
9  define("USER_DB", "root");
10 define("PASS_DB", "12345");
11
12 ?>
```

Di dalam file ini akan dibuat beberapa variable constants untuk digunakan di dalam keseluruhan project framework.

Line 12, Column 3 Spaces: 4 PHP

3 - Create Class Database Connection



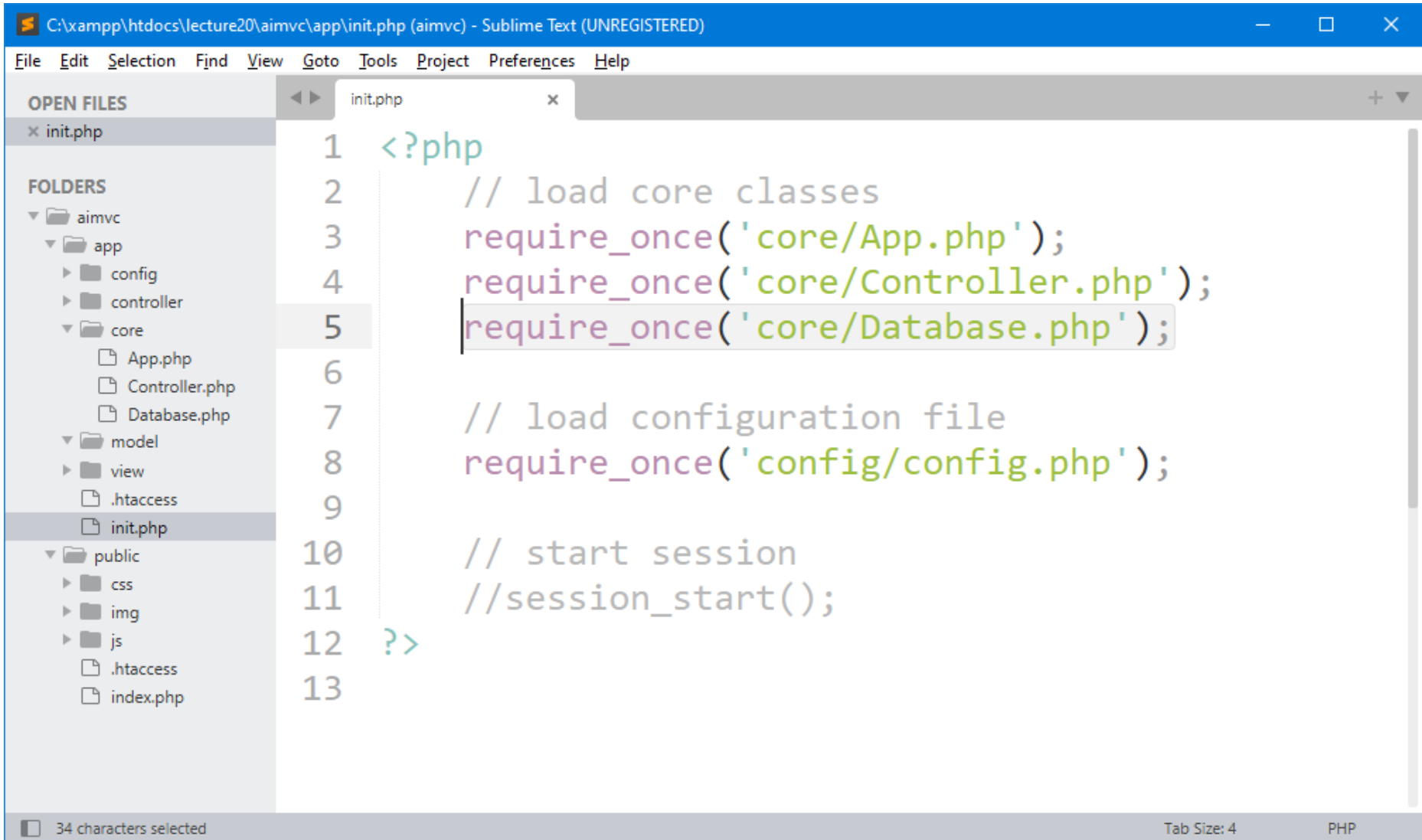
The screenshot shows a Sublime Text editor window titled "C:\xampp\htdocs\lecture20\aimvc\app\core\Database.php (aimvc) - Sublime Text (UNREGISTERED)". The editor displays a PHP class named `Database` with the following code:

```
1 <?php
2 class Database{
3     private $server_name = HOST_DB;
4     private $db_name     = NAME_DB;
5     private $user_name   = USER_DB;
6     private $password    = PASS_DB;
7
8     private $con;
9
10    public function __construct(){
11        // check database connection
12        $this->con = $this->db_connection($this->server_name, $this->user_name, $this->password, $this->db_name);
13        if($this->con==false){
14            // status connection failed.
15        }else{
16            // status connected successfully.
17        }
18    }
19
20    private function db_connection($srvr_nm, $usr_nm, $psswr, $db_nm){
21        // create connection
22        $conn = new mysqli($srvr_nm, $usr_nm, $psswr, $db_nm);
23        // check connection
24        if ($conn->connect_error) {
25            //die("Connection failed: " . $conn->connect_error);
26            return false;
27        }
28        return $conn;
29    }
30
31    public function query($sql){
32        $result = $this->con->query($sql);
33
34        if ($result->num_rows > 0) {
35            // convert to associative array
36            $rows = mysqli_fetch_all($result, MYSQLI_ASSOC);
37            return $rows;
38        } else {
39            return []; // Empty array
40        }
41    }
42
43    public function db_close(){
44        $this->con->close();
45    }
46 }
47 ?>
```

The left sidebar shows the "FOLDERS" panel with a tree view of the project structure:

- aimvc
 - app
 - config
 - controller
 - core
 - App.php
 - Controller.php
 - Database.php**
 - model
 - view
 - .htaccess
 - init.php
 - public
 - css
 - img
 - js
 - .htaccess
 - index.php

4 – Manage Core *Database.php* in *init.php*



C:\xampp\htdocs\lecture20\aimvc\app\init.php (aimvc) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES
x init.php

FOLDERS
▼ aimvc
▼ app
 config
 controller
 core
 App.php
 Controller.php
 Database.php
 model
 view
 .htaccess
 init.php
▼ public
 css
 img
 js
 .htaccess
 index.php

```
1  <?php
2      // load core classes
3      require_once('core/App.php');
4      require_once('core/Controller.php');
5      require_once('core/Database.php');
6
7      // load configuration file
8      require_once('config/config.php');
9
10     // start session
11     //session_start();
12  ?>
13
```

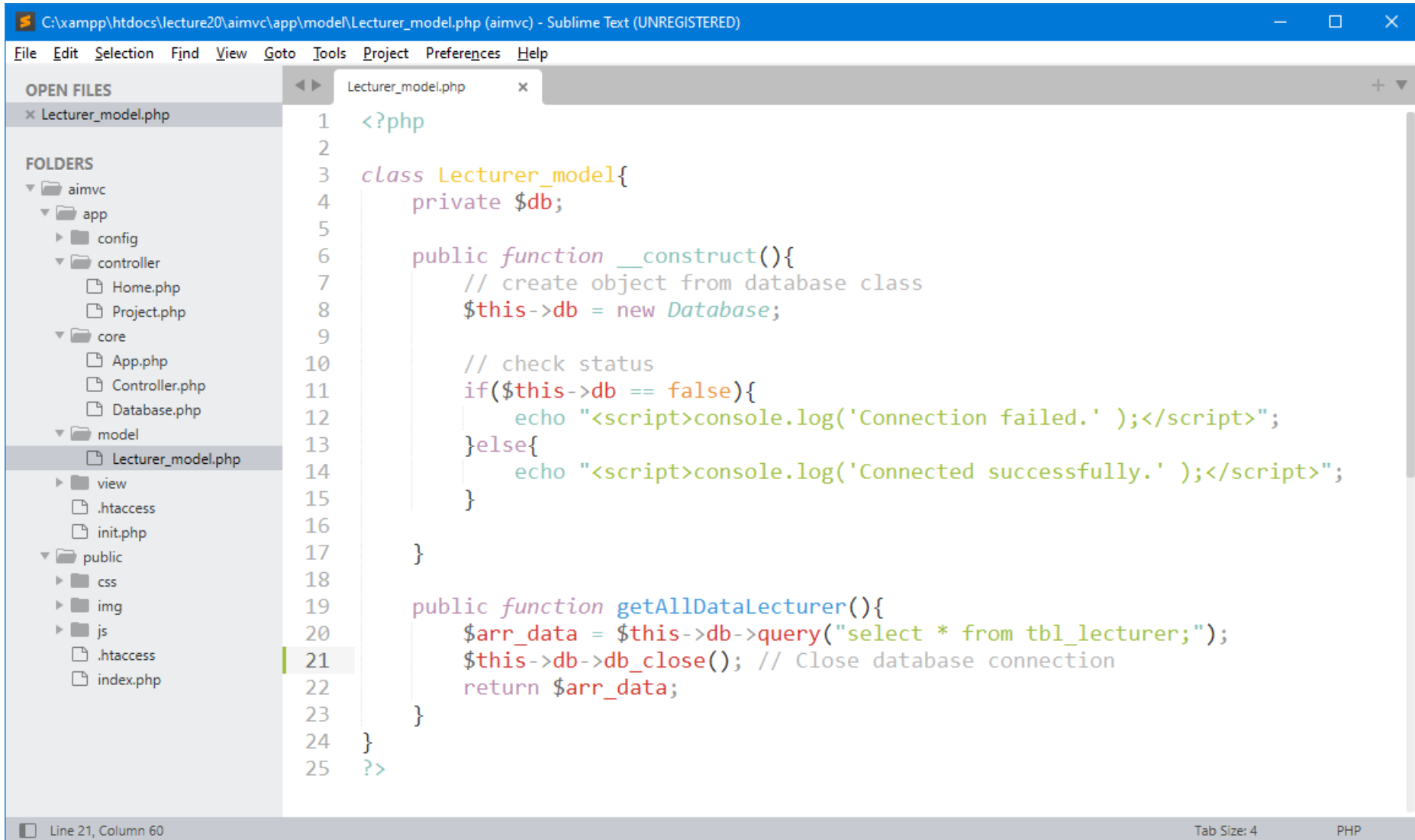
34 characters selected Tab Size: 4 PHP

A decorative border of green leaves and branches at the top of the slide.

2 – Create Component Model [to store data & its related **logic**]

A decorative border of green leaves and branches at the bottom of the slide.

1 – Create Model Component (*Lecturer_model*)



```
C:\xampp\htdocs\lecture20\aimvc\app\model\Lecturer_model.php (aimvc) - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES
x Lecturer_model.php

FOLDERS
aimvc
  app
    config
    controller
      Home.php
      Project.php
    core
      App.php
      Controller.php
      Database.php
    model
      Lecturer_model.php
    view
      .htaccess
      init.php
  public
    css
    img
    js
    .htaccess
    index.php

1  <?php
2
3  class Lecturer_model{
4      private $db;
5
6      public function __construct(){
7          // create object from database class
8          $this->db = new Database;
9
10         // check status
11         if($this->db == false){
12             echo "<script>console.log('Connection failed.' );</script>";
13         }else{
14             echo "<script>console.log('Connected successfully.' );</script>";
15         }
16     }
17
18     public function getAllDataLecturer(){
19         $arr_data = $this->db->query("select * from tbl_lecturer;");
20         $this->db->db_close(); // Close database connection
21         return $arr_data;
22     }
23 }
24 }
25 ?>
```

Line 21, Column 60

Tab Size: 4 PHP

Create model component with ***related logic*** and get the data from database.

2 – How to Read the *data* from Controller?

C:\xampp\htdocs\lecture20\aimvc\app\controller\Home.php (aimvc) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES

× Home.php

FOLDERS

- aimvc
 - app
 - config
 - controller
 - Home.php
 - Project.php
 - core
 - App.php
 - Controller.php
 - Database.php
 - model
 - Lecturer_model.php
 - view
 - .htaccess
 - init.php
 - public
 - css
 - img
 - js
 - .htaccess
 - index.php

```
1 <?php
2 class Home extends Controller{
3     // Constructor
4     public function __construct(){
5
6     }
7
8     // Default method
9     public function index($name="Juan", $age="24"){
10         // Associative Arrays (arrays with keys)
11         $arr_data['name'] = $name;
12         $arr_data['age'] = $age;
13         $arr_data['title'] = "Home Page";
14         $arr_data['lecturers'] = $this->logic("Lecturer_model")->getAllDataLecturer();
15         // Display page and send data
16         $this->display('template/header', $arr_data);
17         $this->display("home/index", $arr_data);
18         $this->display('template/footer');
19     }
20
21     public function page($current_page=2, $next_page=3, $prev_page=1){
22         // Associative Arrays (arrays with keys)
23         $arr_data['current'] = $current_page;
24         $arr_data['next'] = $next_page;
25         $arr_data['previous'] = $prev_page;
26         $arr_data['title'] = "Personal Page";
27         // Display page and send data
28         $this->display('template/header', $arr_data);
29         $this->display("home/page", $arr_data);
30         $this->display('template/footer');
31     }
32
33     // Get status
34     private function _status(){
35         echo "This is home/status.";
36     }
37 }
38 ?>
```

Logic System

Controller

View

Model

MySQL Databases

display() method

logic() method

User Interface

Manage data

MySQL Databases

78 characters selected

3 – Modify Code in /view/home/index.php

C:\xampp\htdocs\lecture20\aimvc\app\view\home\index.php (aimvc) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

OPEN FILES

- index.php

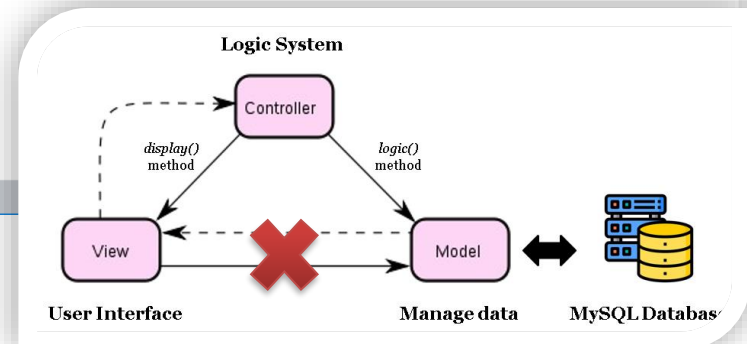
FOLDERS

- aimvc
 - app
 - config
 - controller
 - core
 - model
 - view
 - home
 - index.php
 - page.php
 - project
 - template
 - .htaccess
 - init.php
 - public
 - css
 - img
 - js
 - .htaccess
 - index.php

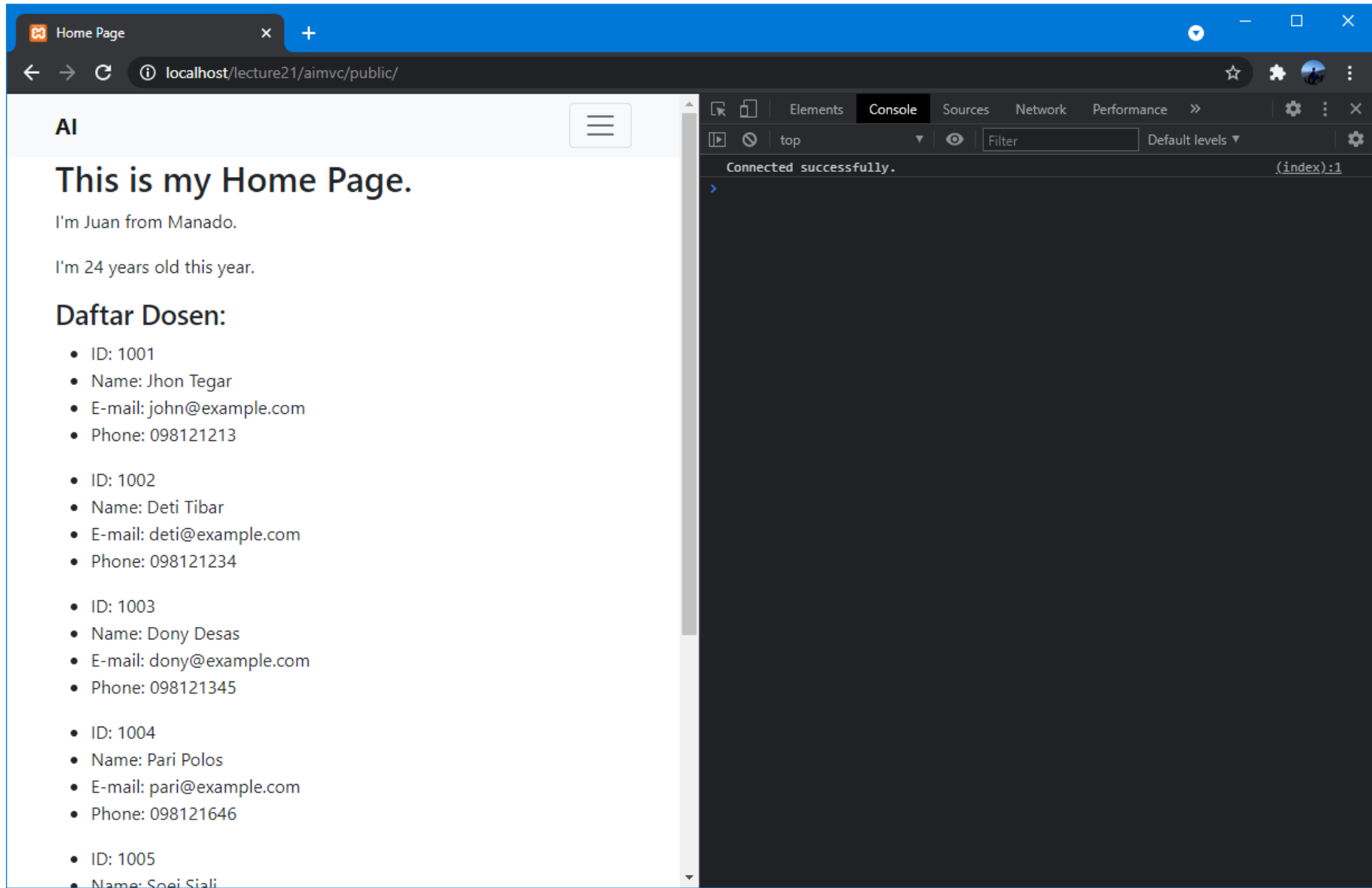
```
1 <div class="container">
2   <h1>This is my Home Page.</h1>
3
4   <p>I'm <?php echo $data['name']; ?> from Manado.</p>
5   <p>I'm <?php echo $data['age']; ?> years old this year.</p>
6   <p>
7     <h3>Daftar Dosen:</h3>
8     <?php foreach($data['lecturers'] as $lecs): ?>
9       <ul>
10        <li> ID: <?php echo $lecs['id']; ?> </li>
11        <li> Name: <?php echo $lecs['name']; ?> </li>
12        <li> E-mail: <?php echo $lecs['email']; ?> </li>
13        <li> Phone: <?php echo $lecs['phone']; ?> </li>
14      </ul>
15    <?php endforeach; ?>
16  </p>
17  
19</div>
```

```
// display method and send any data
public function display($view, $data=[]){
    require_once "../app/view/" . $view . ".php";
}
```

```
// core logic model method
public function logic($model){
    require_once "../app/model/" . $model . ".php";
    $obj_model = new $model;
    return $obj_model;
}
```



4 – Testing View Data *from* Database



The screenshot shows a web browser window with a single tab titled "Home Page". The address bar displays "localhost/lecture21/aimvc/public/". The page content includes a heading "AI", a main heading "This is my Home Page.", and two paragraphs: "I'm Juan from Manado." and "I'm 24 years old this year." Below these is a section titled "Daftar Dosen:" followed by a list of five professors, each with their ID, Name, E-mail, and Phone number.

The Chrome DevTools console is open on the right side of the browser window. The "Console" tab is selected, showing a message "Connected successfully." with a source of "(index):1". The console also displays a blue arrow cursor pointing to the first line of the page content.

AI

This is my Home Page.

I'm Juan from Manado.

I'm 24 years old this year.

Daftar Dosen:

- ID: 1001
- Name: Jhon Tegar
- E-mail: john@example.com
- Phone: 098121213

- ID: 1002
- Name: Deti Tibar
- E-mail: deti@example.com
- Phone: 098121234

- ID: 1003
- Name: Dony Desas
- E-mail: dony@example.com
- Phone: 098121345

- ID: 1004
- Name: Pari Polos
- E-mail: pari@example.com
- Phone: 098121646

- ID: 1005
- Name: Soei Siali



Exercise #1 *for* Students

[PERTAMA]

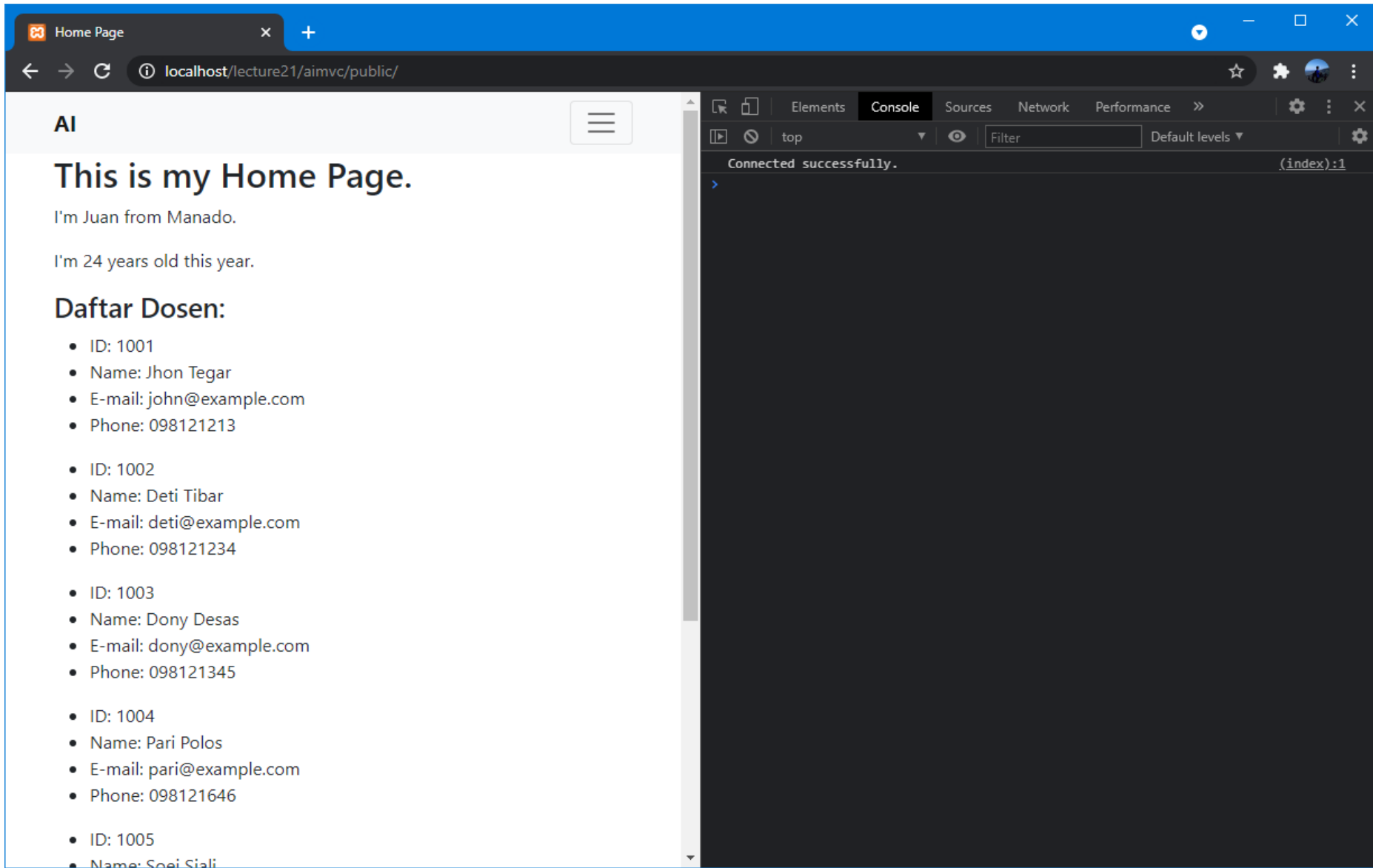
**Lakukan uji coba di
local PC masing-
masing.**

Latihan untuk *Students*

Create directory : C:\xampp\htdocs\lecture25\
Create file : `_.php`

Buat semua contoh yang sudah di pelajari sebelumnya di *lecture* ini.

Expected Output



[SELANJUTNYA]

**Upload code Lecture #25
ke sisi server untuk di
DEMO.**

Create Directory *in* Server

[1] Masuk sebagai *administrator* atau *root*:

➤ ***sudo su***

[2] Buat project folder baru dengan perintah (Parent directory):

➤ ***sudo mkdir -p /var/www/html/lecture25***

[3] Konfigurasi permisi hak akses dan pemilik dari folder:

➤ ***sudo chmod -R 755 /var/www/html/lecture25***

➤ ***sudo chown -R semmy:semmy /var/www/html/lecture25***

[4] Restart Apache Web Server:

➤ ***sudo systemctl restart apache2***

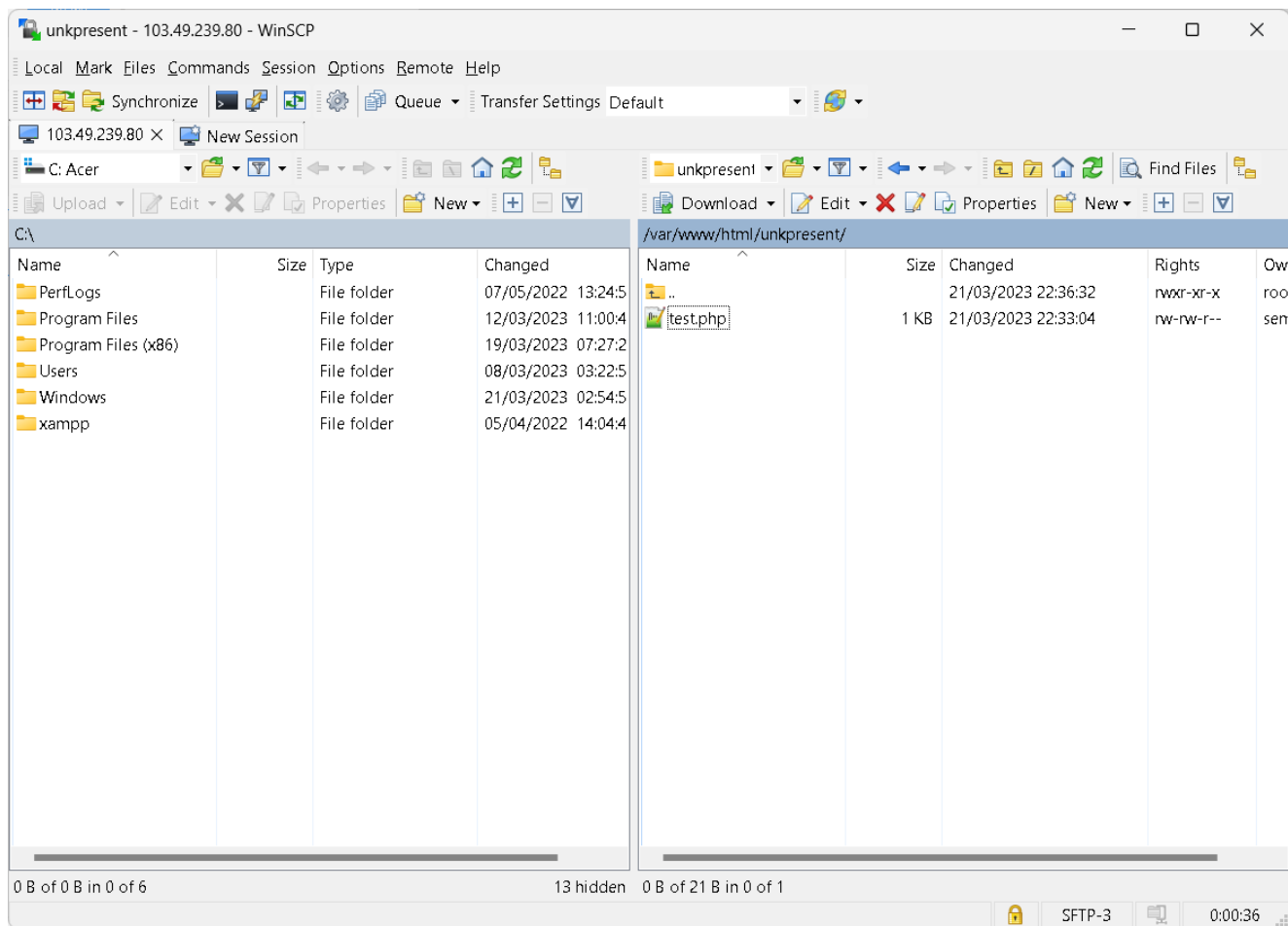
Note:

Ubah nama user “**semmy**” dengan nama *user* access ubuntu kalian.

```
root@unklab: /var/www/html
root@unklab:/home/semmy# sudo mkdir -p /var/www/html/unkpresent
root@unklab:/home/semmy# sudo chmod -R 755 /var/www/html/unkpresent
root@unklab:/home/semmy# cd ..
root@unklab:/home# cd ..
root@unklab:/# ls
bin  dev  home  lib32  libx32  media  opt  root  sbin  srv  tmp  var
boot  etc  lib  lib64  lost+found  mnt  proc  run  snap  sys  usr
root@unklab:/# cd var
root@unklab:/var# ls
backups  crash  local  log  opt  snap  tmp
cache  lib  lock  mail  run  spool  www
root@unklab:/var# cd www/
root@unklab:/var/www# ls
html
root@unklab:/var/www# cd html/
root@unklab:/var/www/html# ls
index.html  test.php  unkpresent
root@unklab:/var/www/html# ls -l
total 20
-rw-r--r-- 1 root root 10671 Mar 26 07:06 index.html
-rw-r--r-- 1 root root 20 Mar 26 07:29 test.php
drwxr-xr-x 2 root root 4096 Mar 26 07:30 unkpresent
root@unklab:/var/www/html# sudo chown -R semmy:semmy /var/www/html/unkpresent
root@unklab:/var/www/html# sudo systemctl restart apache2
root@unklab:/var/www/html# ls -l
total 20
-rw-r--r-- 1 root root 10671 Mar 26 07:06 index.html
-rw-r--r-- 1 root root 20 Mar 26 07:29 test.php
drwxr-xr-x 2 semmy semmy 4096 Mar 26 07:30 unkpresent
root@unklab:/var/www/html#
```

Upload Project via WinSCP

- 1) Buka WinSCP dan arahkan ke “/var/www/html/lecture25”.
- 2) Selanjutnya, upload **file code lecture#25** dari local PC dan ke directory server menggunakan WinSCP.
- 3) Buka via Browser.



Output ditampilkan menggunakan domain name, HTTPS dan bisa di akses secara public via Browser:

Contohnya:

<https://domainkelompok.site/lecture25/aimvc/public/>



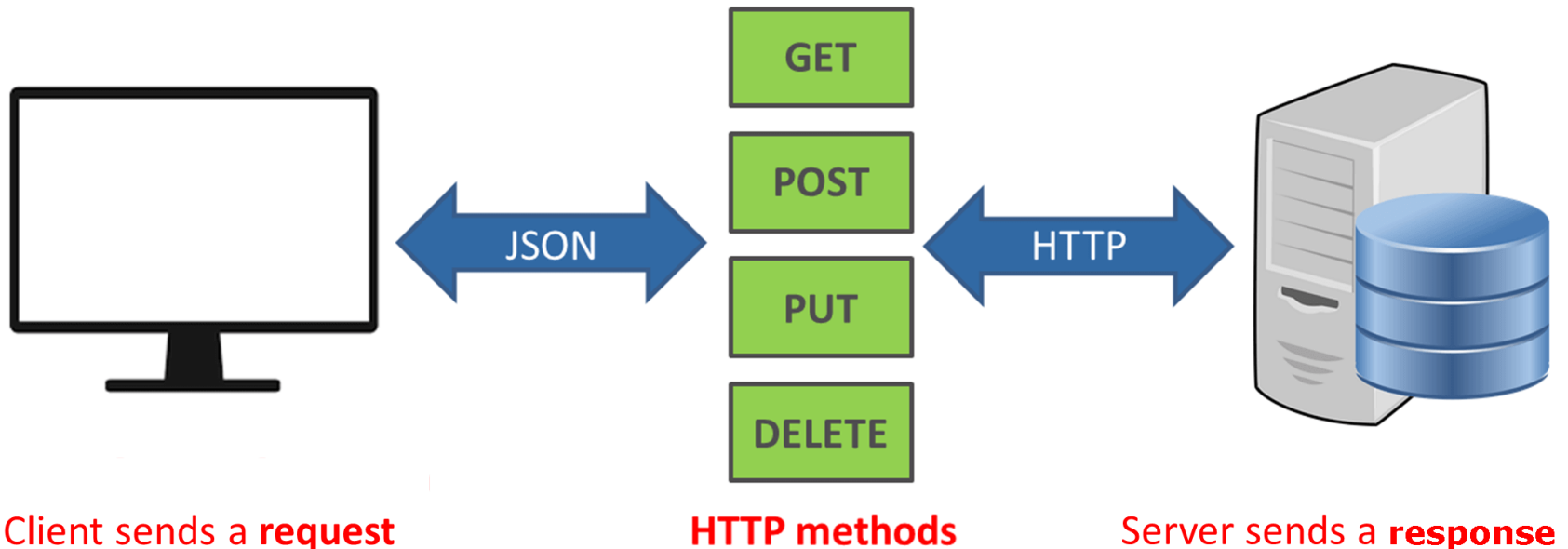
Exercise #2 *for* Students

HTTP Methods

Hypertext Transfer Protocol (HTTP) adalah sebuah jalan atau cara komunikasi dapat terjalin antara *client* (front-end) dengan *server* (back-end).

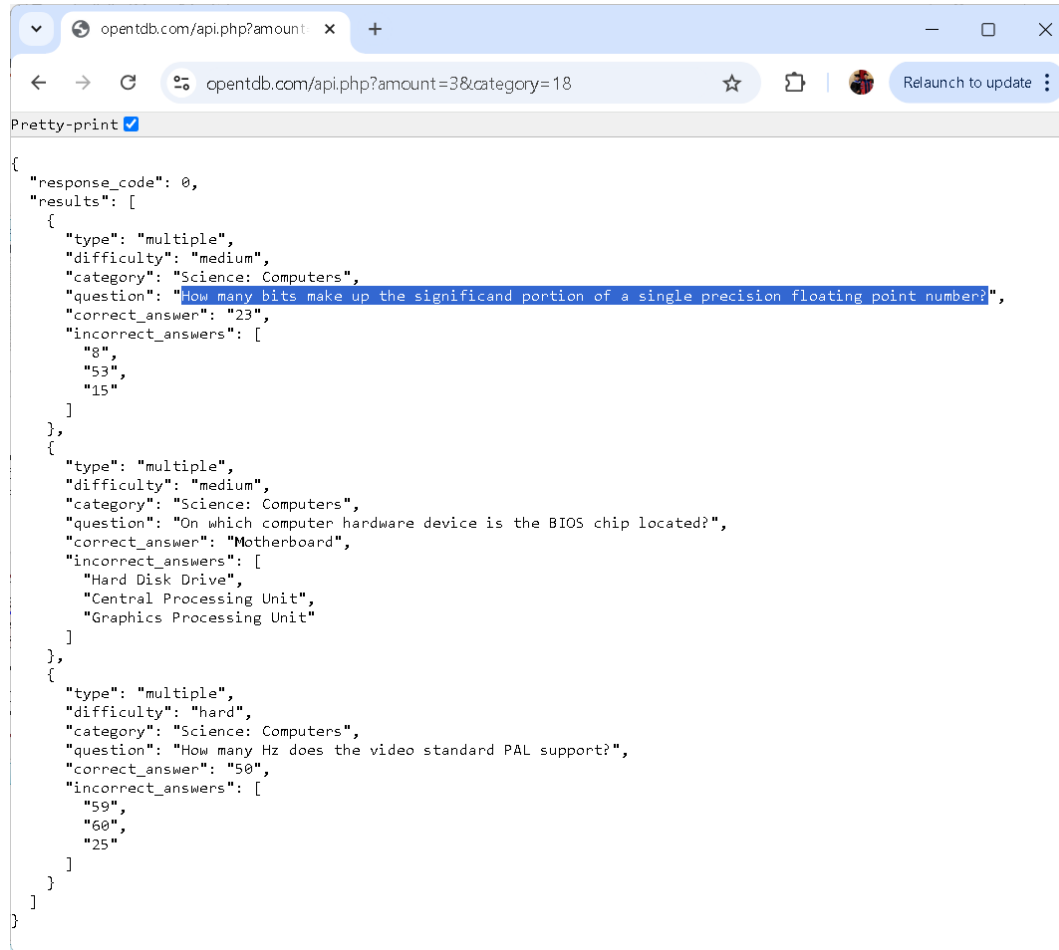
Dalam HTTP Method REST yang sering digunakan antara lain:

- ❖ **GET** : Fungsi ini digunakan untuk perintah menampilkan data.
- ❖ **POST**: Fungsi ini biasanya digunakan untuk perintah dalam menambahkan data (*secure*).
- ❖ **PUT**: Fungsi ini biasanya digunakan untuk perintah edit data.
- ❖ **DELETE**: Fungsi ini biasanya digunakan untuk perintah dalam menghapus data.



RESTful API Web Service

Layanan web RESTful seperti di bawah ini digunakan untuk menyediakan **API web public**.



```
opentdb.com/api.php?amount=3&category=18
Pretty-print
{
  "response_code": 0,
  "results": [
    {
      "type": "multiple",
      "difficulty": "medium",
      "category": "Science: Computers",
      "question": "How many bits make up the significand portion of a single precision floating point number?",
      "correct_answer": "23",
      "incorrect_answers": [
        "8",
        "53",
        "15"
      ]
    },
    {
      "type": "multiple",
      "difficulty": "medium",
      "category": "Science: Computers",
      "question": "On which computer hardware device is the BIOS chip located?",
      "correct_answer": "Motherboard",
      "incorrect_answers": [
        "Hard Disk Drive",
        "Central Processing Unit",
        "Graphics Processing Unit"
      ]
    },
    {
      "type": "multiple",
      "difficulty": "hard",
      "category": "Science: Computers",
      "question": "How many Hz does the video standard PAL support?",
      "correct_answer": "50",
      "incorrect_answers": [
        "59",
        "60",
        "25"
      ]
    }
  ]
}
```

This API is part of the **Open Trivia Database (OpenTDB) API**, which is used to retrieve trivia questions.

The query parameters used in this API call are:

- 1) “amount=1” specifies the number of trivia questions to retrieve (**in this case, 1 question**).
- 2) “category=18” specifies the category of the trivia question. Category 18 corresponds to **“Science: Computers”**.

Keamanan API REST :

- ❖ Manfaatkan HTTPS
- ❖ Gunakan mekanisme otentikasi yang kuat

Pretty-print ☒

```
{
  "response_code": 0,
  "results": [
    {
      "type": "multiple",
      "difficulty": "hard",
      "category": "Science: Computers",
      "question": "Which of these names was an actual codename for a cancelled Microsoft project?",
      "correct_answer": "Neptune",
      "incorrect_answers": [
        "Enceladus",
        "Pollux",
        "Saturn"
      ]
    },
    {
      "type": "multiple",
      "difficulty": "hard",
      "category": "Science: Computers",
      "question": "Which data structure does FIFO apply to?",
      "correct_answer": "Stack",
      "incorrect_answers": [
        "Queue",
        "Heap",
        "Tree"
      ]
    }
  ]
}
```

Four query parameters

What is **Postman**? a popular collaboration platform used for API (Application Programming Interface) web/mobile development?

Postman adalah platform kolaborasi untuk pengembangan API. Fitur Postman menyederhanakan setiap langkah pembuatan API dan menyederhanakan kolaborasi sehingga kita dapat membuat API yang lebih baik dan lebih cepat.

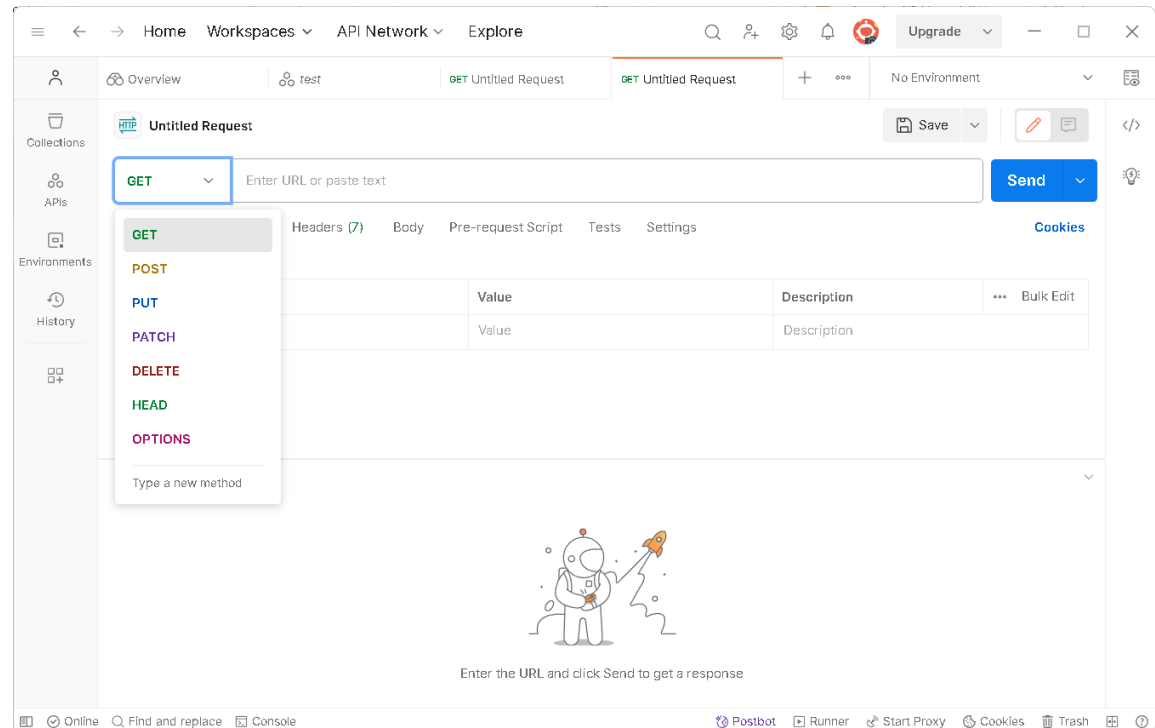
Gunakan URL API yang sudah dibuat (Contoh):

<https://opentdb.com/api.php?amount=2&category=18&difficulty=hard&type=multiple>

Cara download Postman:

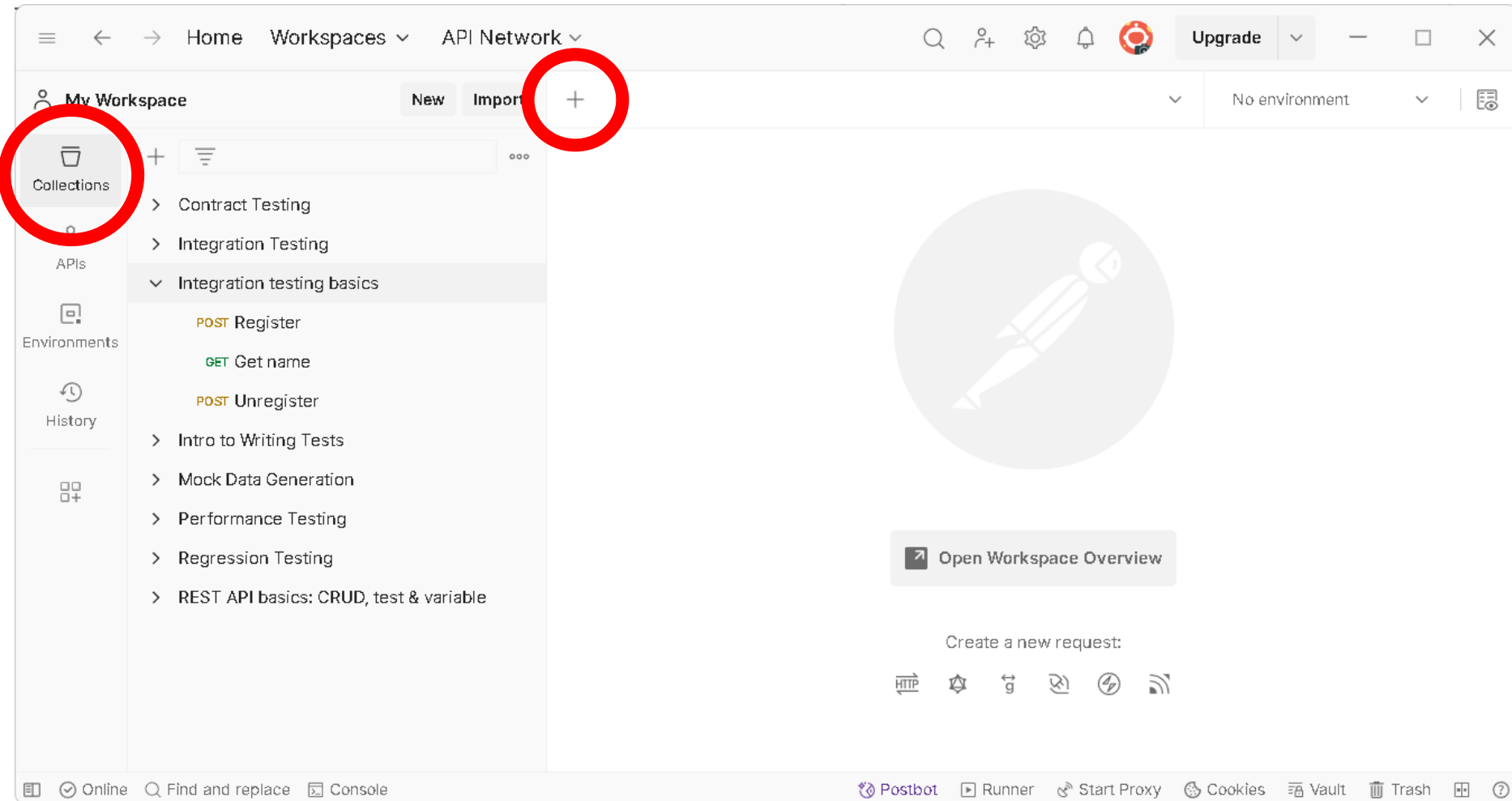
- 1) Buka official website postman di <https://www.postman.com/downloads/>
- 2) Pilih Download App
- 3) Selesai Download Double klik dan pilih install -> Tunggu sampai proses selesai.
- 4) Jika sudah selesai silahkan klik tombol Launch untuk menjalankan
- 5) Silahkan Registrasi menggunakan email aktif

**Sign in dan
Silahkan
Demo cara
menggunakan
an Postman:**



Buka Aplikasi Postman

Buat request API Baru dengan cara di bagian atas Postman, klik tombol **+ New Tab** (tab baru). Ini akan membuka jendela baru untuk membuat request.



Pilih Jenis Server Request

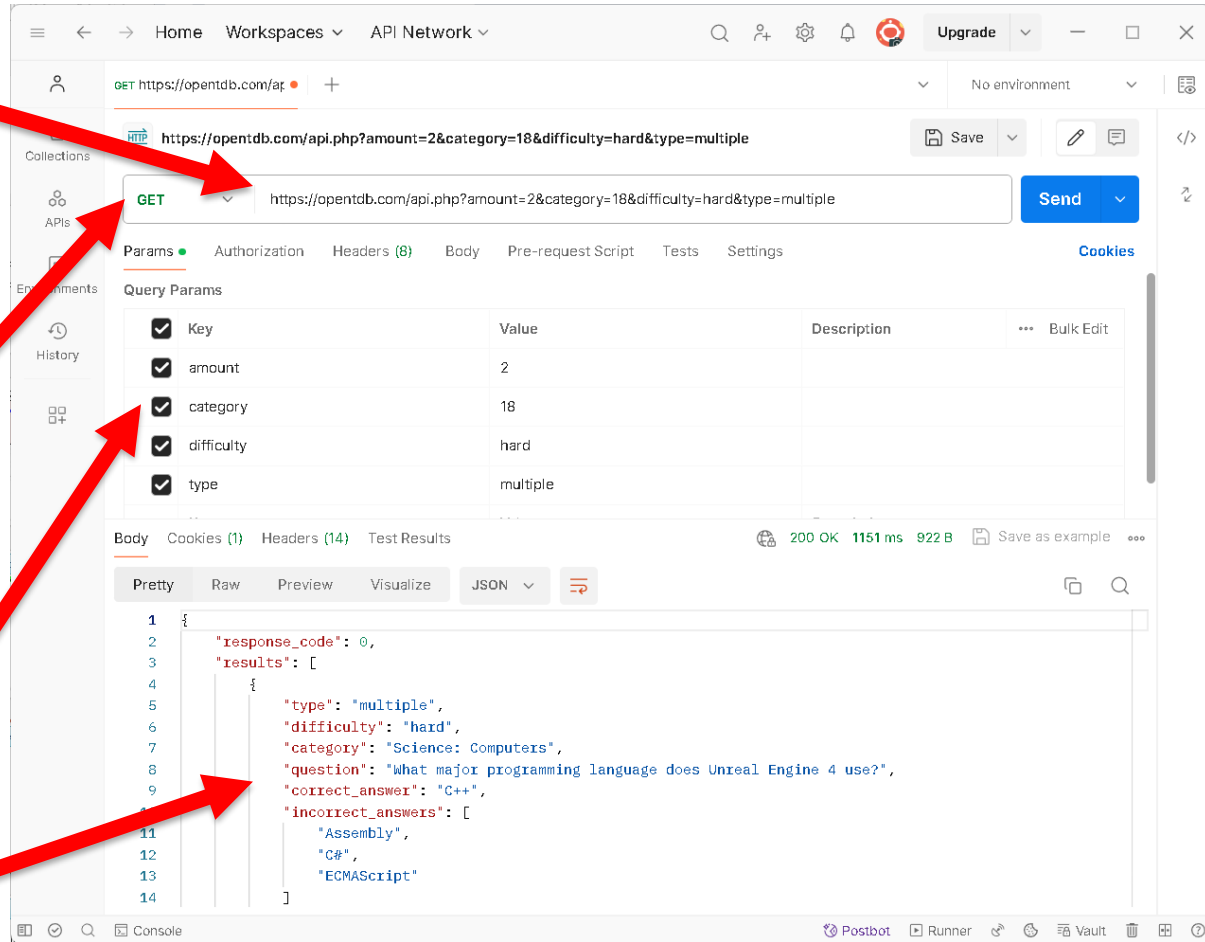
Masukkan **URL Endpoint API**, di sebelah dropdown menu jenis request, masukkan URL endpoint API yang ingin kamu uji.

Klik dropdown dan pilih jenis request yang sesuai:

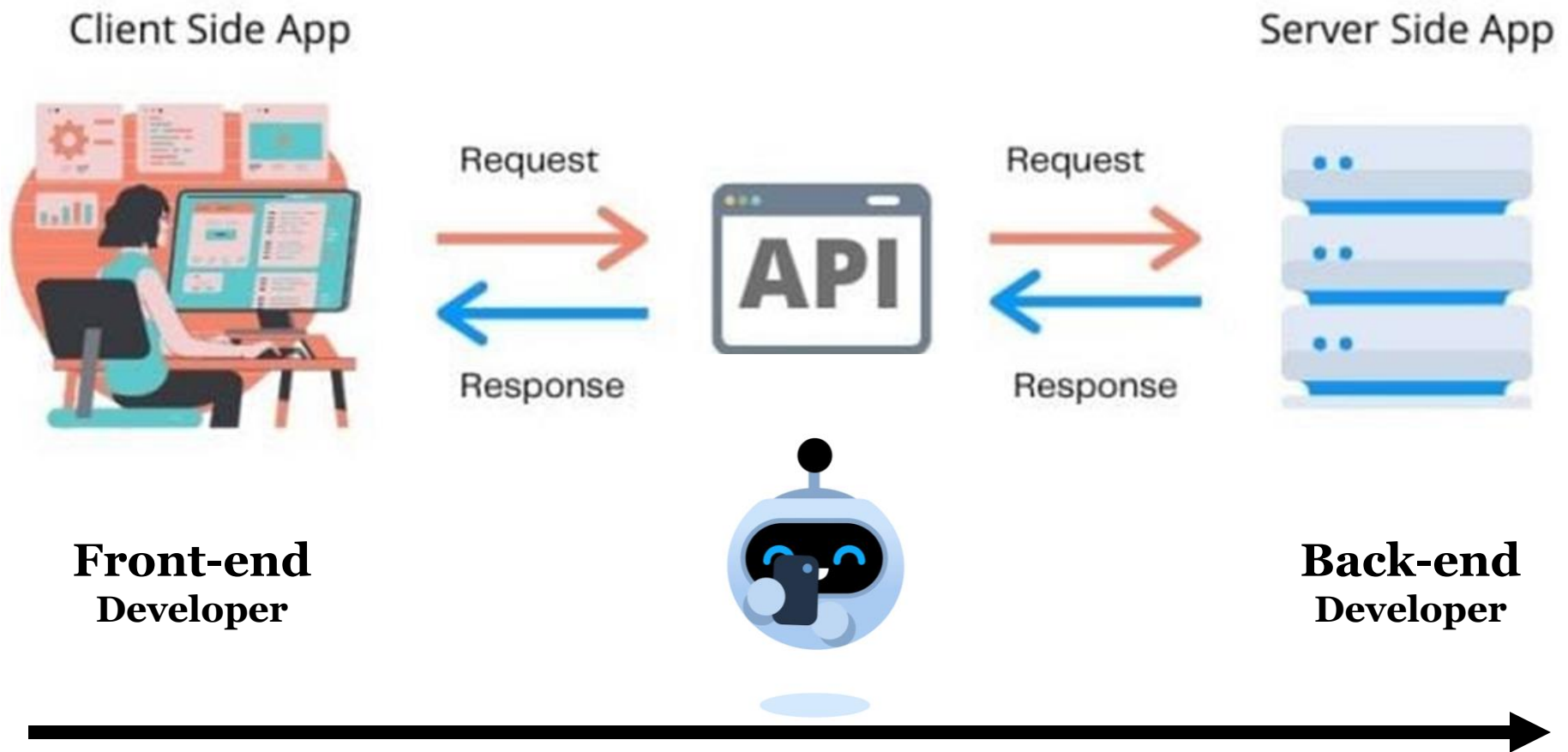
- 1) GET (mengambil data dari server)
- 2) POST (mengirim data ke server)
- 3) PUT (mengubah data di server)
- 4) DELETE (menghapus data di server)

Masukkan **query parameter data** berupa request yang akan dikirimkan ke server. Method **GET** akan secara otomatis jika sudah terdapat key di URL, sedangkan method **POST** harus diinputkan secara manual.

Output Json format adalah respons yang di return oleh server di sisi back-end.



Kapan kita akan menggunakan method POST dan method GET untuk request?

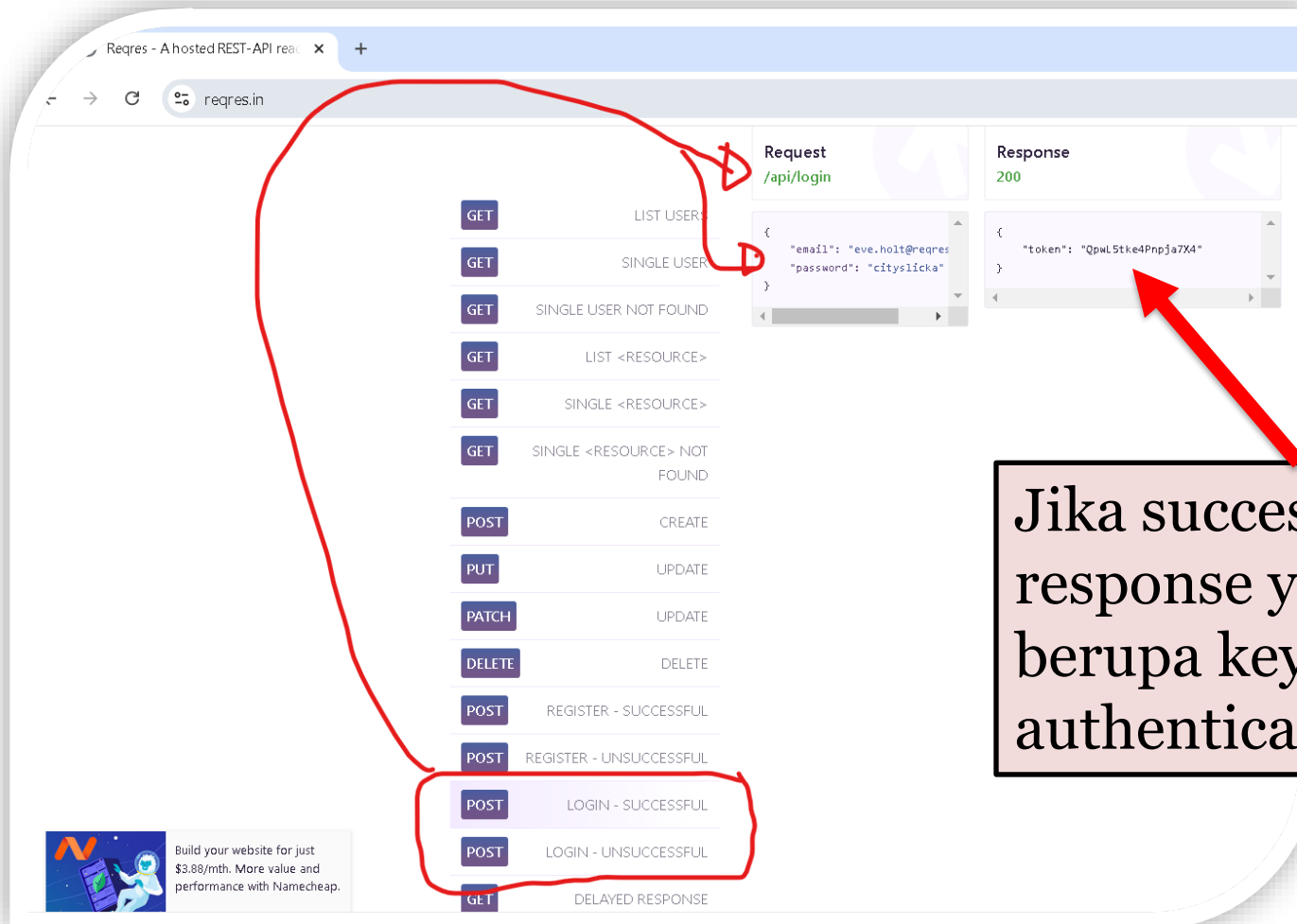


Setiap kali **back-end developer** memberikan URL Endpoint API yang mereka buat untuk front-end, tugas dari **seorang front-end developer** yaitu memastikan setiap **jenis request**, **URL endpoint API** yang ingin diuji dan **data parameter** yang akan di request ke server sesuai atau tidak.

URL Endpoint API *for* Login

Request URL Endpoint API: <https://reqres.in/api/login>

Parameter request: email & password



Jika success, maka output response yang diberikan berupa key session atau authentication key.

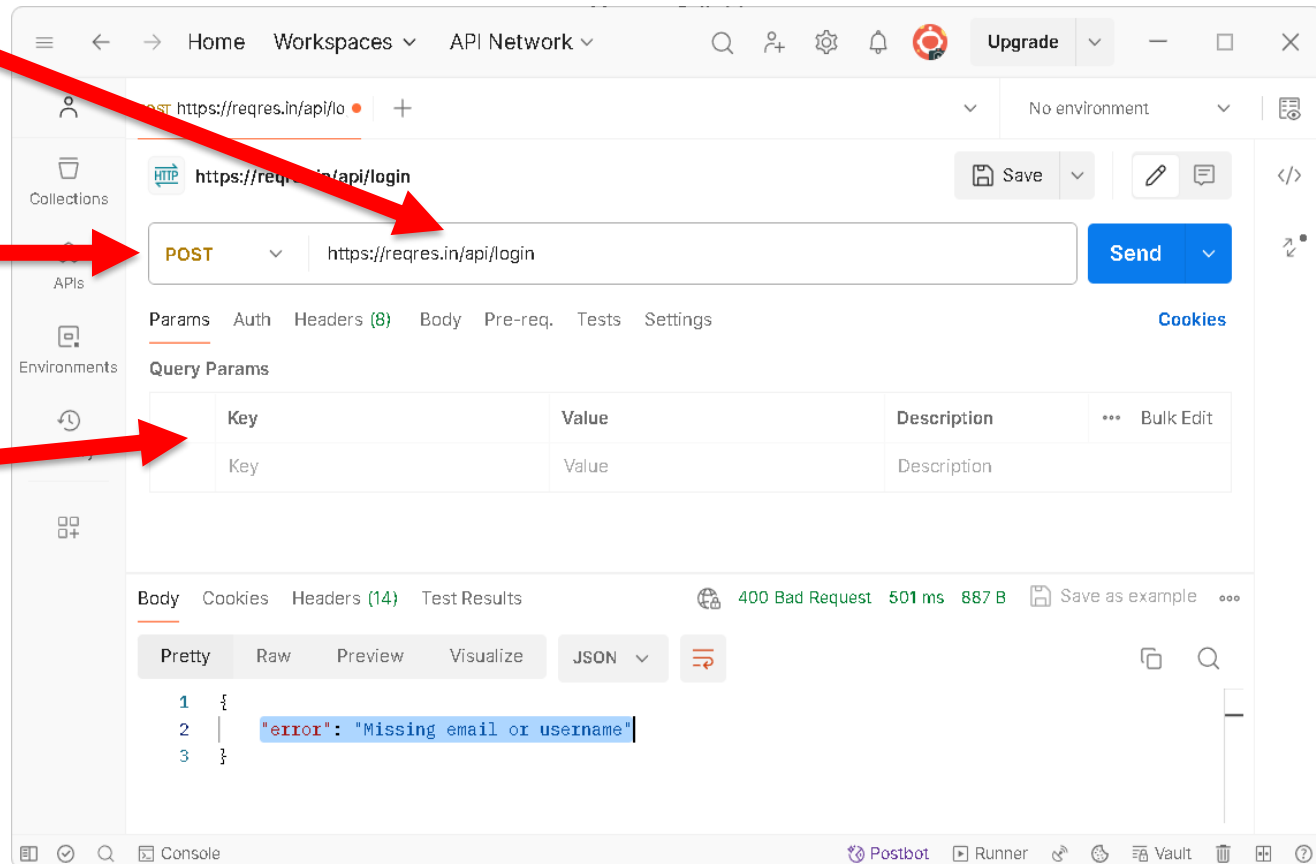
Login Postman Failed

Pilih jenis request, masukan URL endpoint API dan specify query parameter data yang akan di request ke server. Perhatikan bawah, request di bawah tidak ada email dan password yang diinputkan.

URL Endpoint API

Jenis Request

Query Parameter



Login Postman Failed

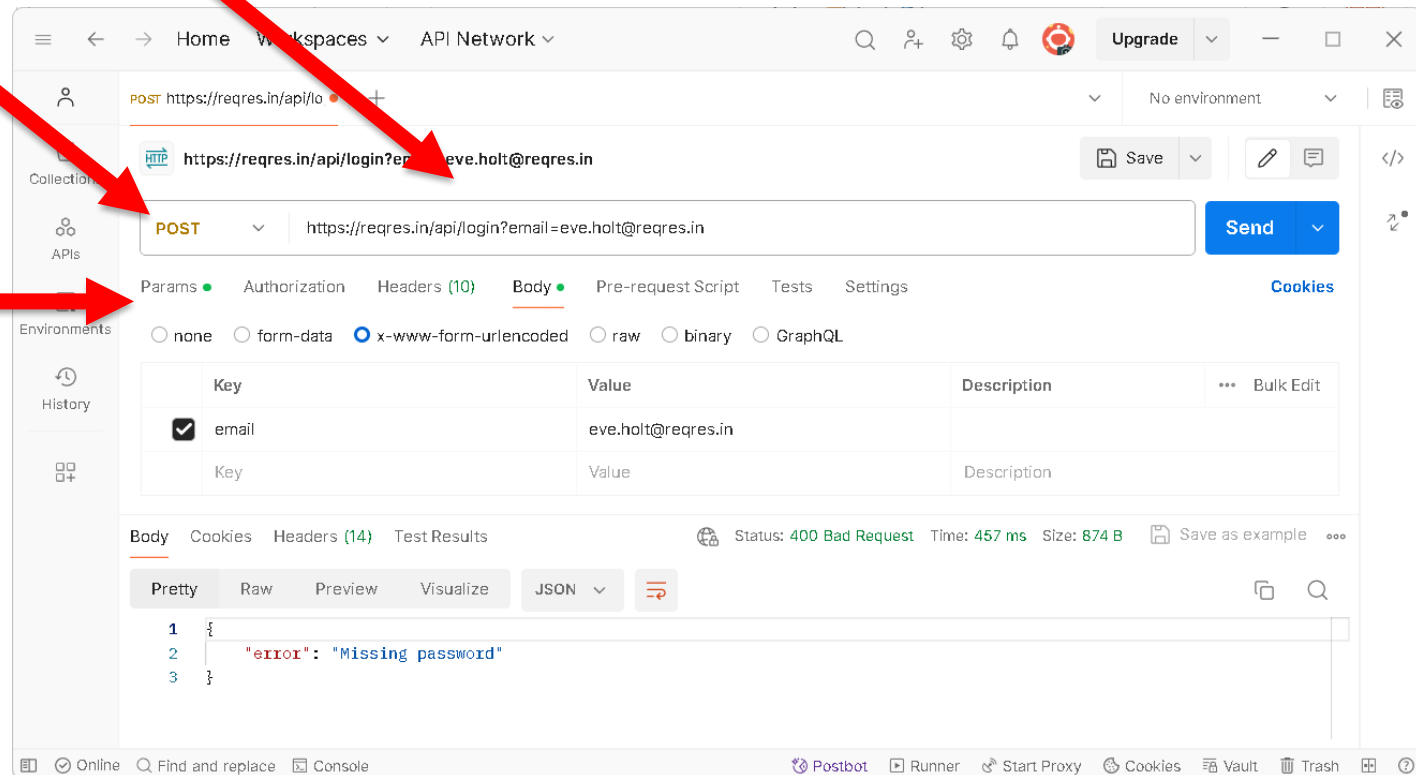
Pilih jenis request, masukan URL endpoint API dan specify query parameter data yang akan di request ke server. Perhatikan bawah, request di bawah tidak ada password yang diinputkan, hanya email saja.

URL Endpoint API

Jenis Request

Query Parameter

Di bagian query parameter pilih “**Body**”, selanjutnya “**x-www-form-urlencoded**”.



Login Postman Success

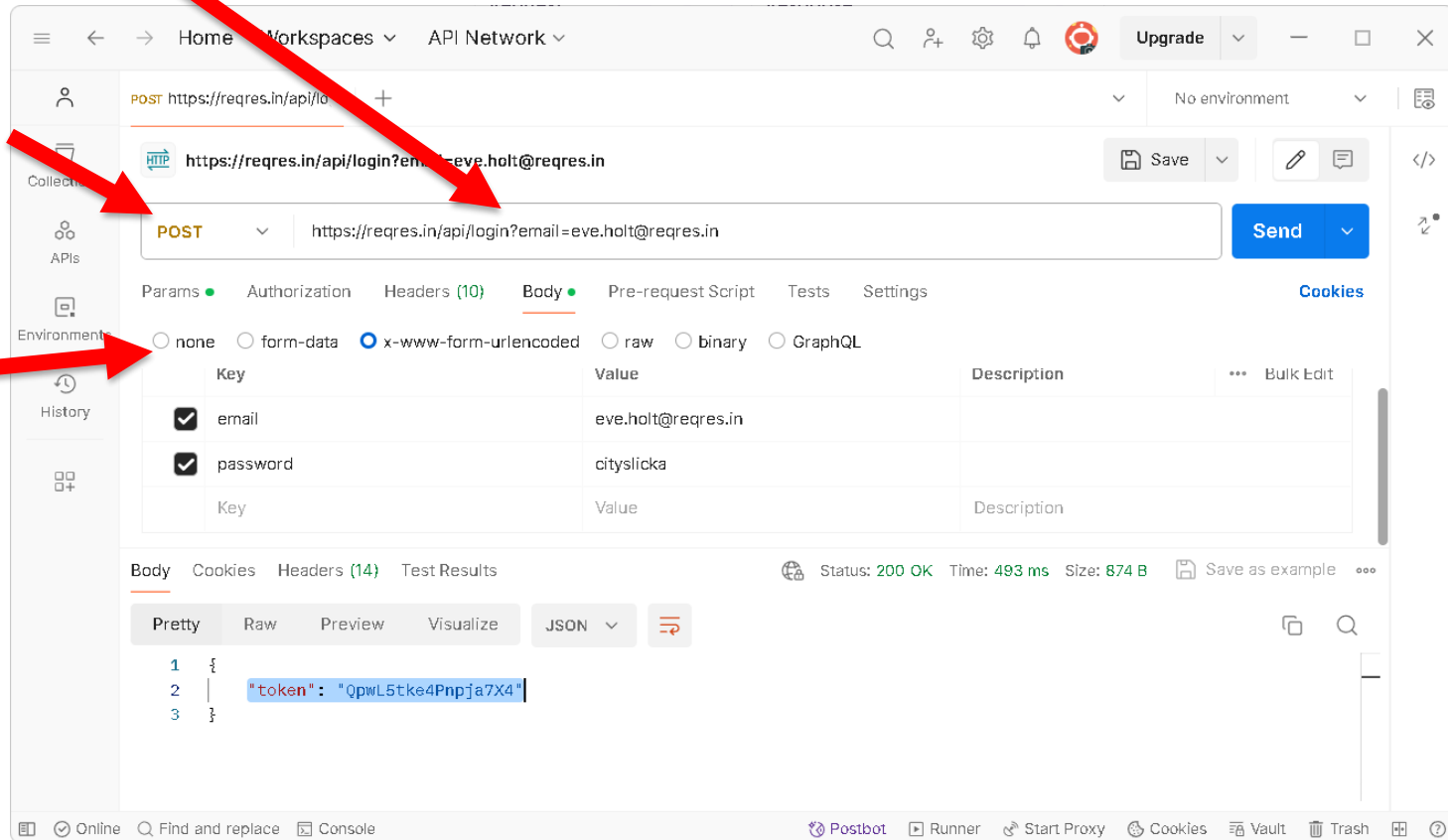
Pilih jenis request, masukan URL endpoint API dan specify query parameter data yang akan di request ke server. Perhatikan bawah, request di bawah sudah terdapat email dan password yang diinputkan.

URL Endpoint API

Jenis Request

Query Parameter

Di bagian query parameter pilih “Body”, selanjutnya “x-www-form-urlencoded”.



END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU



References

XAMPP phpMyAdmin: How to Create Database and Tables

<https://blog.templatetoaster.com/xampp-phpmyadmin/>

Unable to login to phpMyAdmin in XAMPP

<https://techubber.blogspot.com/2017/11/phpmyadmin-access-denied-user-root-no-password-unable-login-phpmyadmin.html>

Creating MySQL Database with XAMPP

<https://www.javatpoint.com/creating-mysql-database-with-xampp>

How to create a database using phpMyAdmin and XAMPP

<https://skillforge.com/how-to-create-a-database-using-phpmyadmin-xampp/>

Mengubah Password Root Pada phpMyAdmin

<https://jagowebdev.com/mengubah-password-root-mysql-dengan-phpmyadmin/>

PHP MySQL Database

https://www.w3schools.com/php/php_mysql_select.asp